

#	Description	Level
9.1.1	Verify that self-contained tokens are validated using their digital signature or MAC to protect against tampering before accepting the token's contents.	1
9.1.2	Verify that only algorithms on an allowlist can be used to create and verify self-contained tokens, for a given context. The allowlist must include the permitted algorithms, ideally only either symmetric or asymmetric algorithms, and must not include the 'None' algorithm. If both symmetric and asymmetric must be supported, additional controls will be needed to prevent key confusion.	1
9.1.3	Verify that key material that is used to validate self-contained tokens is from trusted pre-configured sources for the token issuer, preventing attackers from specifying untrusted sources and keys. For JWTs and other JWS structures, headers such as 'jku', 'x5u', and 'jwk' must be validated against an allowlist of trusted sources.	1

V9.2 Token content

Before making security decisions based on the content of a self-contained token, it is necessary to validate that the token has been presented within its validity period and that it is intended for use by the receiving service and for the purpose for which it was presented. This helps avoid insecure cross-usage between different services or with different token types from the same issuer.

Specific requirements for OAuth and OIDC are covered in the dedicated chapter.

#	Description	Level
9.2.1	Verify that, if a validity time span is present in the token data, the token and its content are accepted only if the verification time is within this validity time span. For example, for JWTs, the claims 'nbf' and 'exp' must be verified.	1
9.2.2	Verify that the service receiving a token validates the token to be the correct type and is meant for the intended purpose before accepting the token's contents. For example, only access tokens can be accepted for authorization decisions and only ID Tokens can be used for proving user authentication.	2
9.2.3	Verify that the service only accepts tokens which are intended for use with that service (audience). For JWTs, this can be achieved by validating the 'aud' claim against an allowlist defined in the service.	2

#	Description	Level
9.2.4	Verify that, if a token issuer uses the same private key for issuing tokens to different audiences, the issued tokens contain an audience restriction that uniquely identifies the intended audiences. This will prevent a token from being reused with an unintended audience. If the audience identifier is dynamically provisioned, the token issuer must validate these audiences in order to make sure that they do not result in audience impersonation.	2

References

For more information, see also:

- OWASP JSON Web Token Cheat Sheet for Java Cheat Sheet (but has useful general guidance)

V10 OAuth and OIDC

Control Objective

OAuth2 (referred to as OAuth in this chapter) is an industry-standard framework for delegated authorization. For example, using OAuth, a client application can obtain access to APIs (server resources) on a user's behalf, provided the user has authorized the client application to do so.

By itself, OAuth is not designed for user authentication. The OpenID Connect (OIDC) framework extends OAuth by adding a user identity layer on top of OAuth. OIDC provides support for features including standardized user information, Single Sign-On (SSO), and session management. As OIDC is an extension of OAuth, the OAuth requirements in this chapter also apply to OIDC.

The following roles are defined in OAuth:

- The OAuth client is the application that attempts to obtain access to server resources (e.g., by calling an API using the issued access token). The OAuth client is often a server-side application.
 - A confidential client is a client capable of maintaining the confidentiality of the credentials it uses to authenticate itself with the authorization server.
 - A public client is not capable of maintaining the confidentiality of credentials for authenticating with the authorization server. Therefore, instead of authenticating itself (e.g., using 'client_id' and 'client_secret' parameters), it only identifies itself (using a 'client_id' parameter).
- The OAuth resource server (RS) is the server API exposing resources to OAuth clients.
- The OAuth authorization server (AS) is a server application that issues access tokens to OAuth clients. These access tokens allow OAuth clients to access RS resources, either on behalf of an

end-user or on the OAuth client's own behalf. The AS is often a separate application, but (if appropriate) it may be integrated into a suitable RS.

- The resource owner (RO) is the end-user who authorizes OAuth clients to obtain limited access to resources hosted on the resource server on their behalf. The resource owner consents to this delegated authorization by interacting with the authorization server.

The following roles are defined in OIDC:

- The relying party (RP) is the client application requesting end-user authentication through the OpenID Provider. It assumes the role of an OAuth client.
- The OpenID Provider (OP) is an OAuth AS that is capable of authenticating the end-user and provides OIDC claims to an RP. The OP may be the identity provider (IdP), but in federated scenarios, the OP and the identity provider (where the end-user authenticates) may be different server applications.

OAuth and OIDC were initially designed for third-party applications. Today, they are often used by first-party applications as well. However, when used in first-party scenarios, such as authentication and session management, the protocol adds some complexity, which may introduce new security challenges.

OAuth and OIDC can be used for many types of applications, but the focus for ASVS and the requirements in this chapter is on web applications and APIs.

Since OAuth and OIDC can be considered logic on top of web technologies, general requirements from other chapters always apply, and this chapter cannot be taken out of context.

This chapter addresses best current practices for OAuth2 and OIDC aligned with specifications found at <https://oauth.net/2/> and <https://openid.net/developers/specs/>. Even if RFCs are considered mature, they are updated frequently. Thus, it is important to align with the latest versions when applying the requirements in this chapter. See the references section for more details.

Given the complexity of the area, it is vitally important for a secure OAuth or OIDC solution to use well-known industry-standard authorization servers and apply the recommended security configuration.

Terminology used in this chapter aligns with OAuth RFCs and OIDC specifications, but note that OIDC terminology is only used for OIDC-specific requirements; otherwise, OAuth terminology is used.

In the context of OAuth and OIDC, the term “token” in this chapter refers to:

- Access tokens, which shall only be consumed by the RS and can either be reference tokens that are validated using introspection or self-contained tokens that are validated using some key material.
- Refresh tokens, which shall only be consumed by the authorization server that issued the token.
- OIDC ID Tokens, which shall only be consumed by the client that triggered the authorization flow.

The risk levels for some of the requirements in this chapter depend on whether the client is a confidential client or regarded as a public client. Since using strong client authentication mitigates many attack vectors, a few requirements might be relaxed when using a confidential client for L1 applications.

V10.1 Generic OAuth and OIDC Security

This section covers generic architectural requirements that apply to all applications using OAuth or OIDC.

#	Description	Level
10.1.1	Verify that tokens are only sent to components that strictly need them. For example, when using a backend-for-frontend pattern for browser-based JavaScript applications, access and refresh tokens shall only be accessible for the backend.	2
10.1.2	Verify that the client only accepts values from the authorization server (such as the authorization code or ID Token) if these values result from an authorization flow that was initiated by the same user agent session and transaction. This requires that client-generated secrets, such as the proof key for code exchange (PKCE) 'code_verifier', 'state' or OIDC 'nonce', are not guessable, are specific to the transaction, and are securely bound to both the client and the user agent session in which the transaction was started.	2

V10.2 OAuth Client

These requirements detail the responsibilities for OAuth client applications. The client can be, for example, a web server backend (often acting as a Backend For Frontend, BFF), a backend service integration, or a frontend Single Page Application (SPA, aka browser-based application).

In general, backend clients are regarded as confidential clients and frontend clients are regarded as public clients. However, native applications running on the end-user device can be regarded as confidential when using OAuth dynamic client registration.

#	Description	Level
10.2.1	Verify that, if the code flow is used, the OAuth client has protection against browser-based request forgery attacks, commonly known as cross-site request forgery (CSRF), which trigger token requests, either by using proof key for code exchange (PKCE) functionality or checking the 'state' parameter that was sent in the authorization request.	2

#	Description	Level
10.2.2	Verify that, if the OAuth client can interact with more than one authorization server, it has a defense against mix-up attacks. For example, it could require that the authorization server return the 'iss' parameter value and validate it in the authorization response and the token response.	2
10.2.3	Verify that the OAuth client only requests the required scopes (or other authorization parameters) in requests to the authorization server.	3

V10.3 OAuth Resource Server

In the context of ASVS and this chapter, the resource server is an API. To provide secure access, the resource server must:

- Validate the access token, according to the token format and relevant protocol specifications, e.g., JWT-validation or OAuth token introspection.
- If valid, enforce authorization decisions based on the information from the access token and permissions which have been granted. For example, the resource server needs to verify that the client (acting on behalf of RO) is authorized to access the requested resource.

Therefore, the requirements listed here are OAuth or OIDC specific and should be performed after token validation and before performing authorization based on information from the token.

#	Description	Level
10.3.1	Verify that the resource server only accepts access tokens that are intended for use with that service (audience). The audience may be included in a structured access token (such as the 'aud' claim in JWT), or it can be checked using the token introspection endpoint.	2
10.3.2	Verify that the resource server enforces authorization decisions based on claims from the access token that define delegated authorization. If claims such as 'sub', 'scope', and 'authorization_details' are present, they must be part of the decision.	2
10.3.3	Verify that if an access control decision requires identifying a unique user from an access token (JWT or related token introspection response), the resource server identifies the user from claims that cannot be reassigned to other users. Typically, it means using a combination of 'iss' and 'sub' claims.	2

#	Description	Level
10.3.4	Verify that, if the resource server requires specific authentication strength, methods, or recentness, it verifies that the presented access token satisfies these constraints. For example, if present, using the OIDC 'acr', 'amr' and 'auth_time' claims respectively.	2
10.3.5	Verify that the resource server prevents the use of stolen access tokens or replay of access tokens (from unauthorized parties) by requiring sender-constrained access tokens, either Mutual TLS for OAuth 2 or OAuth 2 Demonstration of Proof of Possession (DPoP).	3

V10.4 OAuth Authorization Server

These requirements detail the responsibilities for OAuth authorization servers, including OpenID Providers.

For client authentication, the 'self_signed_tls_client_auth' method is allowed with the prerequisites required by section 2.2 of RFC 8705.

#	Description	Level
10.4.1	Verify that the authorization server validates redirect URIs based on a client-specific allowlist of pre-registered URIs using exact string comparison.	1
10.4.2	Verify that, if the authorization server returns the authorization code in the authorization response, it can be used only once for a token request. For the second valid request with an authorization code that has already been used to issue an access token, the authorization server must reject a token request and revoke any issued tokens related to the authorization code.	1
10.4.3	Verify that the authorization code is short-lived. The maximum lifetime can be up to 10 minutes for L1 and L2 applications and up to 1 minute for L3 applications.	1
10.4.4	Verify that for a given client, the authorization server only allows the usage of grants that this client needs to use. Note that the grants 'token' (Implicit flow) and 'password' (Resource Owner Password Credentials flow) must no longer be used.	1

#	Description	Level
10.4.5	Verify that the authorization server mitigates refresh token replay attacks for public clients, preferably using sender-constrained refresh tokens, i.e., Demonstrating Proof of Possession (DPoP) or Certificate-Bound Access Tokens using mutual TLS (mTLS). For L1 and L2 applications, refresh token rotation may be used. If refresh token rotation is used, the authorization server must invalidate the refresh token after usage, and revoke all refresh tokens for that authorization if an already used and invalidated refresh token is provided.	1
10.4.6	Verify that, if the code grant is used, the authorization server mitigates authorization code interception attacks by requiring proof key for code exchange (PKCE). For authorization requests, the authorization server must require a valid 'code_challenge' value and must not accept a 'code_challenge_method' value of 'plain'. For a token request, it must require validation of the 'code_verifier' parameter.	2
10.4.7	Verify that if the authorization server supports unauthenticated dynamic client registration, it mitigates the risk of malicious client applications. It must validate client metadata such as any registered URIs, ensure the user's consent, and warn the user before processing an authorization request with an untrusted client application.	2
10.4.8	Verify that refresh tokens have an absolute expiration, including if sliding refresh token expiration is applied.	2
10.4.9	Verify that refresh tokens and reference access tokens can be revoked by an authorized user using the authorization server user interface, to mitigate the risk of malicious clients or stolen tokens.	2
10.4.10	Verify that confidential client is authenticated for client-to-authorized server backchannel requests such as token requests, pushed authorization requests (PAR), and token revocation requests.	2
10.4.11	Verify that the authorization server configuration only assigns the required scopes to the OAuth client.	2
10.4.12	Verify that for a given client, the authorization server only allows the 'response_mode' value that this client needs to use. For example, by having the authorization server validate this value against the expected values or by using pushed authorization request (PAR) or JWT-secured Authorization Request (JAR).	3
10.4.13	Verify that grant type 'code' is always used together with pushed authorization requests (PAR).	3

#	Description	Level
10.4.14	Verify that the authorization server issues only sender-constrained (Proof-of-Possession) access tokens, either with certificate-bound access tokens using mutual TLS (mTLS) or DPoP-bound access tokens (Demonstration of Proof of Possession).	3
10.4.15	Verify that, for a server-side client (which is not executed on the end-user device), the authorization server ensures that the 'authorization_details' parameter value is from the client backend and that the user has not tampered with it. For example, by requiring the usage of pushed authorization request (PAR) or JWT-secured Authorization Request (JAR).	3
10.4.16	Verify that the client is confidential and the authorization server requires the use of strong client authentication methods (based on public-key cryptography and resistant to replay attacks), such as mutual TLS ('tls_client_auth', 'self_signed_tls_client_auth') or private key JWT ('private_key_jwt').	3

V10.5 OIDC Client

As the OIDC relying party acts as an OAuth client, the requirements from the section “OAuth Client” apply as well.

Note that the “Authentication with an Identity Provider” section in the “Authentication” chapter also contains relevant general requirements.

#	Description	Level
10.5.1	Verify that the client (as the relying party) mitigates ID Token replay attacks. For example, by ensuring that the 'nonce' claim in the ID Token matches the 'nonce' value sent in the authentication request to the OpenID Provider (in OAuth2 refereed to as the authorization request sent to the authorization server).	2
10.5.2	Verify that the client uniquely identifies the user from ID Token claims, usually the 'sub' claim, which cannot be reassigned to other users (for the scope of an identity provider).	2
10.5.3	Verify that the client rejects attempts by a malicious authorization server to impersonate another authorization server through authorization server metadata. The client must reject authorization server metadata if the issuer URL in the authorization server metadata does not exactly match the pre-configured issuer URL expected by the client.	2

#	Description	Level
10.5.4	Verify that the client validates that the ID Token is intended to be used for that client (audience) by checking that the 'aud' claim from the token is equal to the 'client_id' value for the client.	2
10.5.5	Verify that, when using OIDC back-channel logout, the relying party mitigates denial of service through forced logout and cross-JWT confusion in the logout flow. The client must verify that the logout token is correctly typed with a value of 'logout+jwt', contains the 'event' claim with the correct member name, and does not contain a 'nonce' claim. Note that it is also recommended to have a short expiration (e.g., 2 minutes).	2

V10.6 OpenID Provider

As OpenID Providers act as OAuth authorization servers, the requirements from the section “OAuth Authorization Server” apply as well.

Note that if using the ID Token flow (not the code flow), no access tokens are issued, and many of the requirements for OAuth AS are not applicable.

#	Description	Level
10.6.1	Verify that the OpenID Provider only allows values 'code', 'ciba', 'id_token', or 'id_token code' for response mode. Note that 'code' is preferred over 'id_token code' (the OIDC Hybrid flow), and 'token' (any Implicit flow) must not be used.	2
10.6.2	Verify that the OpenID Provider mitigates denial of service through forced logout. By obtaining explicit confirmation from the end-user or, if present, validating parameters in the logout request (initiated by the relying party), such as the 'id_token_hint'.	2

V10.7 Consent Management

These requirements cover the verification of the user's consent by the authorization server. Without proper user consent verification, a malicious actor may obtain permissions on the user's behalf through spoofing or social-engineering.

#	Description	Level
10.7.1	Verify that the authorization server ensures that the user consents to each authorization request. If the identity of the client cannot be assured, the authorization server must always explicitly prompt the user for consent.	2
10.7.2	Verify that when the authorization server prompts for user consent, it presents sufficient and clear information about what is being consented to. When applicable, this should include the nature of the requested authorizations (typically based on scope, resource server, Rich Authorization Requests (RAR) authorization details), the identity of the authorized application, and the lifetime of these authorizations.	2
10.7.3	Verify that the user can review, modify, and revoke consents which the user has granted through the authorization server.	2

References

For more information on OAuth, please see:

- oauth.net
- OWASP OAuth 2.0 Protocol Cheat Sheet

For OAuth-related requirements in ASVS following published and in draft status RFC-s are used:

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens
- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

For more information on OpenID Connect, please see:

- OpenID Connect Core 1.0